

Project Report

FFCS Slot Clash Validator & Timetable Planner

1. Problem Statement

During our FFCS (Fully Flexible Credit System) course registration every semester, we have to manually check if the subjects we want to take are clashing in time slots. For example, if I select a theory subject in slot A1 and a lab in slot L1+L2, they might clash on Monday mornings. Doing this manually on paper is confusing, and if we make a mistake, the registration portal rejects our selection. There is no simple offline tool to let us test and build our timetable safely before the actual FFCS portal opens.

2. Objectives

- To build a simple command-line Java program where students can add the courses they want to take.
- To automatically detect if any selected time slots overlap with each other.
- To make sure the total credits selected don't break the university limit (minimum 16, maximum 27).
- To print a clean timetable grid so students can visually check their weekly schedule.

3. Methodology and Java Concepts Used

I developed this project using only Core Java without any complicated third-party frameworks, so it can run directly from any terminal. The main approach was to divide the code into different packages (Model, Service, and Exception) to keep the architecture organized.

The main Object-Oriented Programming concepts I used are:

- Classes and Objects: I created real-world blueprints for Course, Slot, and Timetable to store the user's data logically.
- Encapsulation: All the variables inside my model classes are kept private to protect the data, and I used public getter methods to access them.
- Custom Exceptions (Inheritance): Instead of using normal system errors, I created my own error classes by inheriting from the base Exception class. If a student tries to add clashing slots, my program throws a custom SlotClashException.
- Collections Framework: I used an ArrayList to keep track of the dynamic list of courses added by the user. I also used a HashMap to store the database of all slot timings (mapping a slot string

like 'A1' to its actual day and time blocks).

- File Handling: I used `BufferedWriter` and `FileWriter` from the `java.io` package to export the final generated timetable into a `.txt` file for easy sharing.

4. Conclusion

This Java application successfully solves a real problem faced by students during course registration. By applying core Object-Oriented Programming concepts and interval-checking logic, the program can efficiently catch time clashes and enforce credit rules. It acts as a perfect local testing sandbox for students to finalize their semester schedule without stress before entering the actual FFCS portal.